
KS Assistant: A Simple AI Agent for Software Engineering

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Large language models can generate code and call tools effectively, yet deploying
2 them as practical software engineering assistants still reveals persistent gaps: finite
3 context windows, errors that can derail entire sessions, agents that get stuck in dead
4 ends, low-quality output, and changes that are difficult to review or revert.

5 We present **KS Assistant**, a software engineering assistant and integrated develop-
6 ment environment (IDE) built on top of the **KS Agent Framework**, a simple AI
7 agent framework of roughly 1,900 lines of code for the core agents. The framework
8 addresses these gaps through a structured system prompt and a five-layer agent
9 hierarchy in which each layer adds one concern: budget-tracked ReAct execution,
10 automatic continuation across sub-sessions via summarization, coding and browser
11 tools with parallel sub-agents, persistent multi-turn chat with history recall, and
12 git worktree isolation so every task runs on its own branch. Both KS Assistant
13 and the KS Agent Framework encode disciplined software engineering practices
14 directly into the agent’s system prompt, including file review, test-first bug fixing,
15 and pre-finish verification.

16 We implemented KS Assistant as a local Visual Studio Code extension that supports
17 browser automation, multimodal input, Docker containers, and web/mobile access.
18 The system prioritizes verification over latency by giving the model time to check
19 its own output with linters, type checkers, and tests before reporting success. The
20 codebase was developed with assistance from KS Assistant over four months,
21 providing a longitudinal self-hosting case study. On Terminal-Bench 2.0, KS
22 Assistant achieves a 62.2% overall pass rate with Claude Opus 4.6 [Anthropic,
23 2026a]; for context, the public leaderboard reports Claude Code at 58% and Cursor
24 Composer 2 at 61.7%. These results are achieved without benchmark-specific
25 prompt tuning or model fine-tuning.

1 Introduction

27 Modern large language models (LLMs), such as Claude Opus 4.7 [Anthropic, 2026b], GPT 5.5 [Ope-
28 nAI, 2026], and Gemini 3.1 [Google DeepMind, 2026], can generate code, reason about software
29 architecture, and use developer tools [Chen et al., 2021, Rozière et al., 2023]. A growing body of
30 work has explored how to harness these capabilities for autonomous software engineering, from
31 single-session agents that resolve GitHub issues [Yang et al., 2024, Wang et al., 2024] to industrial
32 products marketed as AI software and general assistants [GitHub, 2021, Cursor, 2024, Cognition
33 Labs, 2024, Anthropic, 2025, OpenAI, 2025a, OpenClaw AI, 2025]. Yet using an LLM as a practical
34 software engineering assistant still exposes several persistent gaps: context windows are finite, errors
35 can derail an entire session, agents get stuck in dead ends, models generate low-quality output, and
36 generated changes are difficult to review or revert once applied to a live codebase.

37 We propose the *KS Agent Framework*, a compact AI agent framework containing around 1,900
38 lines of code for the core agent implementation. The name “KS” reflects the *Keep Simple* design
39 philosophy: each layer is small, each concern is isolated, and the overall system avoids unnecessary
40 abstraction. We address the above-mentioned gaps through a structured system prompt (Section 4
41 and Appendix A) and a five-layer agent hierarchy in which each layer handles one concern:

- 42 1. **KS Agent** — budget-tracked ReAct [Yao et al., 2023b] loop with native function calling.
- 43 2. **Relentless Agent** — automatic summarization and continuation across sub-sessions.
- 44 3. **Tool Agent** — coding tools, browser automation, and parallel sub-agent execution.
- 45 4. **Chat Agent** — persistent multi-turn chat sessions with history recall.
- 46 5. **Worktree Agent** — git worktree isolation so every task runs on its own branch.

47 We implemented KS Assistant as a Visual Studio Code extension that runs locally. It includes browser
48 support (using open-source Chromium and Playwright), multimodal support, Docker container
49 support, and a mobile/web app. The system is intended to be released as open-source software; the
50 repository link is withheld to respect double-blind review.

51 KS Assistant has been used to develop its own codebase. The KS Agent framework, the agent
52 layers, the VS Code extension, and the system prompt were developed with KS Assistant, which
53 operates in its own repository. This self-hosting process served as a continuous stress test: when
54 the agent introduced a bug that impaired its functionality, the developers asked the agent to analyze
55 the trajectory and code and propose a fix. The five core agent classes remain compact: the KS
56 Agent comprises 413 lines, the Relentless Agent 321 lines, the Tool Agent 322 lines, the Chat Agent
57 125 lines, and the Worktree Agent 704 lines—a total of roughly 1,885 lines of code (excluding empty
58 lines and comments).

59 We prioritize output quality over speed. Using a weaker or cheaper model can force the developer
60 to discard the agent’s work and start over, increasing the total cost. Conversely, giving a frontier
61 model time to validate its own output—running linters, type checkers, and tests before declaring
62 success—can reduce the number of low-quality changes. We expect token costs and inference
63 latencies to continue to fall [Gao et al., 2025], making this verification-first posture increasingly
64 practical.

65 We evaluate on Terminal-Bench 2.0 and achieve a 62.2% overall pass rate using Claude Opus 4.6 [An-
66 thropic, 2026a]; for context, the public leaderboard reports Claude Code at 58% and Cursor Com-
67 poser 2 at 61.7% [Cursor Research, 2026] on the same benchmark (Section 3). These results are
68 achieved without benchmark-specific prompt tuning or model fine-tuning. The results suggest that
69 a compact layered framework, even without trajectory compaction or asynchronous multi-agent
70 orchestration, can be competitive on this benchmark. The self-hosting case study further suggests that
71 conventional software engineering practices can be useful when expressed as concrete instructions
72 for LLM agents.

73 2 Agent architecture

74 The KS Agent Framework was originally built to rapidly prototype and experiment with prompt
75 optimization techniques [Agrawal et al., 2026] and evolutionary algorithms for algorithmic optimiza-
76 tion [Novikov et al., 2025, Algorithmic Superintelligence, 2025]. The emphasis on simplicity made
77 the framework easier for coding agents to inspect and modify. Eventually, no prompt optimization
78 techniques were used when creating the system prompt for KS Assistant; it was hand-tuned based
79 on long-term experience with the agent and its behavior. The framework uses five agent layers
80 combining composition and inheritance. Each layer delegates upward for concerns it does not own.

81 2.1 KS Agent

82 The KS Agent is the innermost execution unit implementing a standard ReAct loop [Yao et al.,
83 2023b].

```

from ks.core.ks_agent import KSAgent

def calculate(expression: str) -> str:
    """Evaluate a math expression."""
    return str(eval(expression))

agent = KSAgent(name="Math Buddy")
result = agent.run(
    model_name="gemini-2.5-flash",
    prompt_template="Calculate: {question}",
    arguments={"question": "15% of 847?"},
    tools=[calculate]
)
print(result) # 127.05

```

Listing 1: A complete KS agent with a single tool.

Table 1: Terminal-Bench 2.0 aggregate results (89 tasks, 5 trials each, Claude Opus 4.6).

Metric	Value
Total tasks	89
Overall pass rate	62.2% (277/445)
pass@any (1/5 passes)	78.7% (70/89)
pass@all (5/5 pass)	43.8% (39/89)
Always-fail tasks	19
Always-pass tasks	39
Mixed-result tasks	31
Median cost per trial	\$0.45
Mean cost per trial	\$0.90
Median duration / trial	202 s
Mean duration / trial	446 s

Native function calling. Tools are registered as ordinary Python callables. The agent builds an OpenAI-compatible tool schema once at setup time and caches it. A special `finish` tool signals task completion and returns the result. Using a model’s native calling API avoids the mistakes models make with custom function-calling conventions.

Step, token, and budget tracking. At every step, the agent extracts input and output token counts from the API response, computes dollar cost using a per-model pricing table, and updates both local and global budget counters protected by a class-level lock. Three limits are checked before each step: the per-agent budget, the global budget, and the maximum step count.

Error resilience. The agent retries transient API errors (rate limits, server errors) up to a configurable threshold. Non-retryable errors (authentication failures, permission denials) are raised immediately.

Non-agentic mode. When tools are not needed, the agent runs a single generation without the ReAct loop, useful for summarization sub-tasks.

Listing 1 shows a complete, working agent in under ten lines of code. The developer defines an ordinary Python function (`calculate`), instantiates a `KSAgent`, and calls its `run` method with a model name, a prompt template, template arguments, and a list of tools. The framework automatically handles tool-schema generation, the ReAct loop, and budget tracking.

The KS Agent is stateless across runs: each call resets the conversation, token counters, and tool registry, making it safe to reuse a single instance for multiple sequential tasks.

2.2 Relentless Agent

The Relentless Agent wraps a KS Agent in a continuation loop. Its core contribution is executing tasks that exceed a single context window by breaking them into sub-sessions.

Rather than investing in context-compaction techniques, we adopt a simple continuation protocol: when a sub-session exhausts its context window or step budget, the agent produces a *structured summary* of every action taken so far—chronologically ordered, with explanations and relevant code snippets—and a fresh sub-session resumes from that summary. This approach is related to Reflexion [Shinn et al., 2023], which feeds verbal self-critiques back into subsequent trials, but uses a Reflexion-like technique to *continue* a task rather than retry it. We found that a naïve instruction to “summarize the current context” produced poor continuations; requiring a *step-by-step chronological account with code snippets* improved coherence across sub-sessions in our development use. A potential limitation is that summaries may grow unwieldy for multi-day tasks; in practice, we have not encountered this problem even for tasks spanning several hours, but a thorough evaluation of summary scaling remains future work.

Continuation protocol. The `finish` tool accepts three fields: a success flag, a continue flag, and a summary. When `is_continue=True`, the Relentless Agent starts a new sub-session with a fresh context window. The prompt for the new session includes a chronologically ordered list of all prior attempt summaries, instructs the agent not to redo completed work, and advises it to step back and rethink the strategy from scratch if it has been retrying the same approach without progress.

122 **Forced continuation on failure.** If a sub-session raises an exception (e.g., the step limit is hit before
123 calling `finish`), the Relentless Agent saves the full trajectory to a temporary file, spawns a separate
124 summarizer agent to produce a concise summary, and uses that summary as the progress text for the
125 next sub-session. This ensures that even crashed sessions contribute useful context. The summarizer
126 is instructed to read the (potentially large) trajectory file and return a precise, chronologically ordered
127 list of the agent’s actions, along with the reason for each action and relevant code snippets.

128 **Pre-emptive continuation.** To force the agent to self-continue before exhausting its step budget, the
129 system prompt is augmented with a step-threshold instruction that requires the agent, at a designated
130 step or whenever the task is at risk of running out of steps or context length, to call `finish` with
131 `success=False`, `is_continue=True`, and a precise chronologically-ordered summary of work
132 done so far. This instruction is injected near the end of the budget window. Without it, the agent
133 exhibits a “rush to finish” behavior when steps are running low—skipping verification, making hasty
134 edits, and calling `finish` with an incomplete result. The explicit step threshold redirects that urgency
135 into the continuation protocol, ensuring that a clean handoff to a new sub-session produces better
136 results than a frantic attempt to squeeze everything into the remaining steps.

137 2.3 Tool Agent

138 The Tool Agent adds the tools that make the system useful for software development and general-
139 purpose automation.

140 **Coding tools.** Four core tools: a shell command executor with streaming output, a file reader, a
141 precise string-based file editor, and a file writer. The shell executor supports configurable timeouts,
142 streams output in real time, and respects a stop event for user cancellation.

143 **Browser automation.** A web-use tool provides programmatic browser control: navigating to URLs,
144 reading page accessibility trees, clicking elements, typing text, pressing keys, scrolling, and taking
145 screenshots. It uses the open-source Chromium browser via the Playwright library.

146 **Parallel sub-agents.** An optional parallel execution tool spawns independent Tool Agent instances
147 in a thread pool. Each sub-agent gets its own LLM context and tool set, useful for embarrassingly
148 parallel tasks such as summarizing multiple files or researching independent topics. Results are
149 collected and returned in input order. This tool is not enabled by default because the IDE cannot
150 coherently stream multiple agents’ outputs in the chat window.

151 **User interaction.** An ask-user-question tool pauses execution and requests clarification from the
152 user.

153 **Docker isolation.** When a Docker image is specified, coding tools are replaced with Docker-aware
154 variants that execute commands inside a container.

155 2.4 Chat Agent

156 The Chat Agent adds multi-turn conversation persistence.

157 **Chat sessions.** Each task is assigned to a chat session identified by a stable chat ID. Tasks and results
158 are persisted to a local database. New tasks within the same session include prior tasks and results as
159 numbered context entries, allowing the LLM to reference earlier work.

160 **Bounded chat context.** The agent caps in-context history entries at $K = 10$. When the cap is
161 exceeded, it preserves the first two entries (which establish the user’s intent) and the most recent
162 entries, dropping the middle entries least likely to be referenced.

163 **Session management.** Three operations are supported: starting a new chat, resuming by task
164 description, and resuming by explicit chat ID.

165 **Frequent task tracking.** Each time a task is executed, the agent records the task description in
166 a frequency table. The IDE sidebar surfaces the most frequent tasks so users can re-issue them
167 with one click, turning recurring requests (“run the test suite,” “regenerate the changelog”) into a
168 click-to-replay experience.

169 **Metadata persistence.** After each task, the agent records metadata including the model used,
170 working directory, software version, token counts, cost, and whether the task used parallel execution
171 or worktree isolation. This audit trail supports cost analysis and debugging.

172 2.5 Worktree Agent

173 The Worktree Agent is the outermost layer. Its defining feature is git-worktree isolation.

174 **Branch-per-task.** When a task starts, the agent creates a new git branch and a corresponding worktree
175 directory. All agent modifications happen inside the worktree; the user’s main working tree remains
176 untouched.

177 **Dirty-state preservation.** Uncommitted changes in the main working tree are copied into the
178 worktree with a baseline commit. During merge, cherry-pick from the baseline replays only the
179 agent’s changes.

180 **Concurrency safety.** A per-repository file lock serializes checkout, stash, merge, and pop operations.
181 Thread-local storage isolates per-task state.

182 **Crash recovery.** All worktree state is stored in git itself (branch names, git config entries) rather than
183 sidecar files. On process restart, the agent queries git and reconstructs instance attributes, enabling
184 seamless recovery.

185 **Graceful fallback.** If the working directory is not inside a git repository, has no commits, or has a
186 detached HEAD, the agent falls back to direct execution without worktree isolation.

187 3 Evaluation on Terminal-Bench 2.0

188 We evaluate on Terminal-Bench 2.0,¹ a benchmark comprising 89 diverse terminal-based program-
189 ming tasks, ranging from building legacy compilers and configuring servers to solving cryptanalysis
190 challenges and training machine-learning models. Each task runs in an isolated Docker container;
191 a separate verifier automatically judges the result. We use the Harbor² framework to orchestrate
192 execution and Claude Opus 4.6 [Anthropic, 2026a] as the underlying LLM. We do not modify the
193 general system prompt or inject benchmark-specific instructions. Evaluation was carried out on a
194 2025 MacBook Air 15” with an M4 processor and 24 GB RAM.

195 3.1 Setup

196 We run 5 independent trials per task. The agent is a thin Harbor adapter that installs and invokes the
197 KS Assistant CLI inside each container. We hard-skip 9 tasks verified to be infeasible for Opus 4.6
198 across 6+ prior attempts (e.g., CompCert compilation, Windows 3.11 GUI installation, video OCR)
199 to save time and token cost. Skipped tasks still count as failures.

200 3.2 Aggregate results

201 Table 1 (shown earlier alongside Listing 1) summarizes the aggregate statistics.

202 The 62.2% overall pass rate is close to or above two public leaderboard entries: at the time of
203 writing, Claude Code (also Opus 4.6) scores approximately 58% on the Terminal-Bench 2.0 leader-
204 board, and Cursor’s Composer 2—a custom fine-tuned model trained with large-scale reinforcement
205 learning [Cursor Research, 2026]—achieves 61.7%. Because we did not run these systems under a
206 controlled protocol, these numbers should be interpreted as leaderboard context rather than a causal
207 comparison.

208 3.3 Task-level breakdown

209 **Consistently solved tasks (39 of 89).** These include cryptanalysis (FEAL differential), game-playing
210 (chess best move), git operations (leak recovery), server configuration (gRPC key-value store, PyPI
211 server, NGINX logging), data processing (resharding), formal verification (Coq plus_comm), ML

¹<https://www.tbench.ai/>

²<https://github.com/harbor-framework/harbor>

inference (HuggingFace model serving, LLM batching scheduler), and system emulation (QEMU startup). The breadth of this set indicates that the benchmark successes are not concentrated in a single task category.

Consistently failed tasks (19 of 89). The failures cluster into three categories: (1) tasks requiring graphical or multimedia capabilities unavailable in the container (video processing, Windows 3.11 GUI installation, MTEB leaderboard scraping), (2) tasks demanding very long or resource-intensive builds that exceed time or memory limits (CompCert compilation, Doom for MIPS, Caffe CIFAR-10, training fastText on Yelp data), and (3) tasks with niche domain-specific requirements that the model struggles to satisfy (DNA insertion, OCaml GC patching, polyglot C/Python binaries, protein assembly, cell segmentation).

Mixed-result tasks (31 of 89). Tasks such as write-compressor (3/5), crack-7z-hash (4/5), and feal-linear-cryptanalysis (4/5) succeed in most trials but occasionally fail due to non-determinism in the model’s reasoning or timing-sensitive environment interactions. Conversely, cancel-async-tasks (1/5) and dna-assembly (1/5) succeed rarely, suggesting they are at the boundary of the model’s capability.

Leaderboard context. KS Assistant does not score as high as some agents on the Terminal-Bench 2.0 leaderboard. We report the result because the evaluation used the general system prompt and Claude Opus 4.6 without benchmark-specific prompt tuning or model fine-tuning. Leaderboard comparisons should be interpreted cautiously: recent analyses report benchmark-gaming and harness-leakage problems on several agent benchmarks, including Terminal-Bench 2.0 [Stein et al., 2026a,b, Wang et al., 2026].

4 The system prompt

The system prompt is a structured document that governs the agent’s behavior across all tasks. It is not a generic instruction to “be helpful” but rather a precise specification of an engineering discipline. We present the two most distinctive aspects here; the complete prompt is detailed in Appendix A.

4.1 Execution mindset

The prompt opens with two directives that define the expected execution style:

```
# FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL.
# BE RELENTLESS. BE CALM. BE RIGOROUS. BE ACCURATE. CHECK FACTS. NO AI SLOP.
```

Each directive addresses a specific failure mode observed in LLM-based agents:

“FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL.” LLMs have a tendency to drift: they comment on the difficulty of a problem, explore tangential concerns, or ask clarifying questions that the user has already answered. This directive anchors the model on the task at hand and discourages meta-commentary that consumes tokens without making progress.

“BE RELENTLESS.” When an agent encounters an error—a failing test, a type violation, a command that returns unexpected output—the default LLM behavior is to apologize, summarize the failure, and ask the user what to do next. This directive instructs the model to treat errors as obstacles to overcome, not as reasons to stop.

“BE CALM.” The complement of relentlessness. An overly aggressive agent may thrash between approaches without deliberation. This directive encourages the model to analyze errors methodically before reacting.

“BE RIGOROUS.” This instructs the model to follow the verification and testing disciplines encoded later in the prompt, rather than taking shortcuts when a solution *looks* right.

“BE ACCURATE.” LLMs frequently generate plausible-but-wrong code, file paths, or command-line flags. This directive raises the model’s threshold for asserting facts, encouraging it to verify claims against the actual file system or documentation rather than relying on parametric memory.

“CHECK FACTS.” A more specific version of “be accurate,” targeting information the agent collects via web tools.

262 “**NO AI SLOP**.” “AI slop” refers to the low-quality, generic, hedging text that LLMs produce when
263 they lack confidence: filler phrases like “certainly!”, unnecessary caveats, and boilerplate explanations.
264 This directive encourages the model to be concise and substantive.

265 These two sentences define a behavioral contract used throughout later prompt sections. During
266 development, we observed failures that corresponded to these directives (e.g., stopping after the
267 first error when the persistence instruction was absent), but we have not yet performed a controlled
268 ablation of each phrase.

269 4.2 Web research protocol

270 When the agent needs external knowledge, the prompt prescribes a structured research workflow
271 rather than allowing ad-hoc browsing:

```
272 ## Web Research (MANDATORY)
273
274 - **Visit >=30 websites every search. Hard requirement --- don't stop before 30, or rationalize fewer.**
275 - Procedure:
276   1. Create PWD/tmp/information-{unique_id}.md: '# Web Research --- Websites visited: 0/30'
277   1. Per site, append: '## [N/30] URL' + extracted info. Update the header counter on each visit.
278   1. **Don't proceed until counter >=30.**
279   1. If results dry up, try different queries, synonyms, official docs, GitHub repos/issues, Stack Overflow
280     , blogs, Reddit, papers, API refs.
281   1. After 30, review and think deeply.
282 - Ask the user for login help when needed.
283
```

285 The rationale is a two-phase separation between *collection* and *synthesis*. LLMs tend to anchor on
286 the first few results they encounter, biasing their solutions toward a narrow slice of the design space.
287 By forcing the agent to accumulate a broad set of information into a file *before* reasoning about it, the
288 protocol counteracts anchoring bias and encourages the model to consider diverse approaches. The
289 “visit ≥ 30 websites” threshold is a conservative guardrail: it discourages the agent from stopping the
290 research phase after two or three hits, and the resulting information file serves as an auditable artifact
291 of what the agent considered.

292 This two-phase collect-then-synthesize discipline is also applied to local file browsing (Ap-
293 pendix A.11): the agent writes a structured summary of each file’s relevant information into a
294 temporary markdown file, externalizing its understanding into a compact artifact that persists across
295 context boundaries. Analysis happens in the second phase, when the agent reads its own summary
296 and reasons about the collected information as a whole.

297 5 VS Code extension features

298 We release our system as a VS Code extension and a web app. The extension emphasizes several
299 mechanisms that are less central in many existing AI coding assistants, including persistent preference
300 files, real-time budget accounting, and integrated browser automation.

301 **Cross-session self-improvement.** The extension maintains a `USER_PREFS.md` file that the agent
302 reads at the start of each task and *updates* at the end. When the agent discovers a new preference or
303 project invariant during task execution, it writes it to the file. The agent also resolves conflicts: if a
304 new preference contradicts an existing one, the old entry is removed. Over time, the file accumulates
305 project-specific knowledge that can be reused in later sessions. This mechanism is detailed in
306 Appendix A.8.

307 **Real-time budget accountability.** The extension displays real-time cost tracking in the sidebar:
308 input tokens, output tokens, cache hits, dollar cost, and elapsed time are updated at every agent step.
309 Both per-task and global budget ceilings are enforced.

310 **Integrated browser automation.** The extension includes a browser automation tool that allows the
311 agent to navigate URLs, read accessibility trees, click elements, type text, press keys, scroll, and take
312 screenshots—all controlled programmatically from within a VS Code task.

6 Conversational development case study

During development, we used natural-language interaction to inspect and revise nontrivial subsystems. One recurring workflow was to ask KS Assistant for a detailed, step-by-step description of a buggy workflow or algorithm, then issue a follow-up request that revised specific steps in natural language. We illustrate this with a development session drawn from the project’s own history, in which the worktree merge workflow (Section 2.5) was first described and then redesigned through conversational prompts. The session comprises four consecutive tasks; we reproduce the prompts verbatim and summarize the agent’s responses.

Step 1: Understanding the existing workflow. The developer begins by asking the agent to explain the current post-task git lifecycle:

```
Can you tell me what happens, step by step, with git in worktree_Assistant_agent.py when a task finishes?
```

The agent reads the source code and returns a structured summary of the four-phase lifecycle: (1) during `run()`, a new branch and worktree are created and the task executes inside the worktree; when the task completes, *nothing is committed or merged*—the result is returned with merge instructions appended and the worktree stays pending; (2) `merge()` calls `_finalize_worktree()`, which stages all changes, generates a commit message via the LLM, commits, removes the worktree, checks out the original branch, and runs `git merge`; (3) `discard()` removes the worktree, prunes, checks out the original branch, and deletes the task branch; (4) in CLI mode an interactive prompt forces the user to choose [c]ommit and merge or [d]iscard before exiting. The agent also notes a key design invariant: nothing auto-merges; auto-commit occurs only at merge/finalize time; all steps are idempotent; and state can be recovered from git on restart.

Step 2: Simplifying the workflow via natural language. Armed with the workflow description, the developer decides the three-way choice (auto-merge, manual merge, discard) is unnecessarily complex and issues a redesign request:

```
Can you change worktree_Assistant_agent.py and the extension so that after the agent finishes its task, it simply asks "Commit and Merge" or "Discard"? When "Commit and Merge" is clicked by the user, the agent must commit the changes with a generated commit message, merge the branch with the original branch, and delete the worktree and the branch associated with the worktree. If the user clicks "discard", it must delete the worktree and the branch, and checkout the original branch.
```

The agent modifies six files across Python and TypeScript: it updates `discard()` to check out the original branch before deleting the task branch, removes the `manual_merge()` method entirely, simplifies `merge_instructions()` to show only two options, updates the webview UI to replace the three-button toolbar with a two-button “Commit and Merge or Discard?” bar, removes the `manual` action type from the TypeScript type definitions, and removes the corresponding handler from the Python backend. Three tests for the deleted manual-merge path have been removed, and one routing test has been updated. All 28 worktree tests pass after the change.

We omit the remaining two tasks from the main text for space; they appear in Appendix B.

7 Related work

Code-specialized language models. Code Llama [Rozière et al., 2023] fine-tunes Llama 2 for code generation and infilling. StarCoder [Li et al., 2023] trains on permissively licensed GitHub code with a fill-in-the-middle objective, and DeepSeek-Coder [Guo et al., 2024] uses a 2-trillion-token code-and-language corpus with repository-level context. Frontier systems increasingly target agentic software engineering directly: Claude Opus 4.6/4.7 [Anthropic, 2026a,b], GPT 5.5 [OpenAI, 2026], Kimi K2.5 [Kimi Team, 2026], and GLM-5.1 [Z.ai, 2026] emphasize long-horizon coding, tool use, and autonomous task execution. KS Assistant is model-agnostic, so these improvements enter through the backend model rather than through changes to the agent architecture.

Code generation agents and IDEs. SWE-Agent [Yang et al., 2024] and OpenHands [Wang et al., 2024] provide LLM agents for repository-level software tasks such as resolving GitHub issues, typically within one execution session. Agentless [Xia et al., 2024] shows that a simple localize-then-repair pipeline can compete with more autonomous loops on SWE-bench. Devin [Cognition Labs, 2024], Claude Code [Anthropic, 2025], Codex [OpenAI, 2025a], Aider [Gauthier, 2023], GitHub Copilot [GitHub, 2021], Cursor [Cursor, 2024], and Windsurf [Codeium, 2024] expose different combinations of shell, editor, browser, cloud sandbox, and git integration. Cursor Composer 2 [Cursor Research, 2026] further couples an IDE workflow with a reinforcement-learning-trained coding model. KS Assistant differs by making continuation, budget accounting, and branch-per-task worktree isolation first-class layers rather than product-specific behaviors.

Reasoning, tool use, and multi-agent orchestration. ReAct [Yao et al., 2023b], chain-of-thought prompting [Wei et al., 2022], Tree of Thoughts [Yao et al., 2023a], and Reflexion [Shinn et al., 2023] structure reasoning, search, and verbal self-critique. Our Relentless Agent borrows the idea of feeding natural-language artifacts into later attempts, but uses chronological summaries to *continue* a partially completed task after context or step exhaustion, not merely to retry from scratch. ChatDev [Qian et al., 2024], MetaGPT [Hong et al., 2024], and AutoGen [Wu et al., 2023] model development as conversations among role-specialized agents. We instead keep a single primary agent with broad tools and optional parallel sub-agents for independent research or summarization subtasks, avoiding the coordination overhead of always-on role play.

Agent frameworks and context engineering. LangChain [LangChain, 2022], DSPy [Khattab et al., 2024], CrewAI [CrewAI, Inc., 2024], smolagents [Roucher et al., 2025], the OpenAI Agents SDK [OpenAI, 2025b], and Google’s ADK [Google, 2025] offer general agent abstractions such as tool registries, handoffs, guardrails, tracing, and prompt optimization. They are broad infrastructure libraries; KS Assistant is a narrow software-engineering system in which each layer owns one operational concern: budget tracking, continuation, tools, chat memory, or git isolation. Recent work on agentic software engineering, autonomous coding agents, agentic programming, persistent context files, and context engineering [Hassan et al., 2025, Li et al., 2025, Wang et al., 2025, Chatlatanagulchai et al., 2025, Mohsenimofidi et al., 2025] argues that reliable agents need curated task context and controllability. Our system prompt, `USER_PREFS.md`, and per-repository override files instantiate this pattern in a lightweight form.

Benchmarks, self-improvement, and test-time compute. HumanEval [Chen et al., 2021], SWE-bench [Jimenez et al., 2024], LiveCodeBench [Jain et al., 2024], and SWE-bench Pro [Deng et al., 2025] evaluate coding at increasing levels of repository and temporal complexity; Prathifkumar et al. [2025] warns that benchmark contamination can inflate apparent progress. Terminal-Bench 2.0 complements these benchmarks by stressing terminal operation, system configuration, long builds, and tool use. Robeyns et al. [2025] studies agents that improve their own scaffolding; KS Assistant uses a lighter self-improvement loop that accumulates preferences and project invariants across sessions. Gao et al. [2025] studies test-time compute scaling for software engineering, while Get Shit Done [TÂCHES, 2025] uses phased planning documents to fight context rot; both motivate our emphasis on verification, explicit handoff summaries, and disciplined pre-finish checks.

8 Conclusion

A layered, single-concern architecture, combined with a system prompt that encodes an engineering discipline, addresses several practical challenges in deploying LLM agents for software development. On Terminal-Bench 2.0, our system reaches 62.2% (277/445) with Claude Opus 4.6 [Anthropic, 2026a], without benchmark-specific tuning, fine-tuning, or reinforcement learning. This result is competitive with two public leaderboard entries, but should not be read as a controlled head-to-head comparison. The self-hosting case study suggests that conventional software engineering practices can be useful when expressed as concrete instructions an LLM can follow.

Limitations. The evaluation uses one frontier model and one benchmark; the Terminal-Bench 2.0 numbers should therefore be read as evidence for this configuration, not as a universal ranking of agent architectures. Leaderboard comparisons are contextual rather than controlled head-to-head experiments, and the 9 hard-skipped tasks count as failures but still reflect manual infeasibility judgments. The benchmark also underrepresents several real IDE concerns: human review quality, merge-conflict frequency over long projects, privacy-sensitive repositories, GUI-heavy workflows,

419 and collaboration among multiple developers. The Relentless Agent can lose information when a
420 long trajectory is compressed into summaries, and we have not measured how this degrades over
421 multi-day tasks. Worktree isolation requires git and adds merge overhead; non-git projects fall back
422 to direct execution. Finally, the verification-first design spends more tokens and wall-clock time than
423 latency-oriented assistants, and it depends on the availability, cost, and reliability of closed frontier
424 models.

425 References

- 426 Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi,
427 Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis,
428 Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform
429 reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. Oral. arXiv
430 preprint arXiv:2507.19457.
- 431 Algorithmic Superintelligence. OpenEvolve: Open-source implementation of AlphaEvolve. <https://github.com/algorithmicsuperintelligence/openevolve>, 2025.
- 433 Anthropic. Claude Code: Anthropic’s agentic coding system. <https://www.anthropic.com/product/claude-code>, 2025.
- 435 Anthropic. Introducing Claude Opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>, 2026a.
- 436 Anthropic. Introducing Claude Opus 4.7. <https://www.anthropic.com/news/claude-opus-4-7>, 2026b.
- 437 Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Lee-
438 laprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida.
439 Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884*,
440 2025.
- 441 Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé Pinto, Jared Kaplan, Harri Edwards,
442 Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv*
443 *preprint arXiv:2107.03374*, 2021.
- 444 Codeium. Windsurf: The AI-powered IDE. <https://windsurf.com>, 2024.
- 445 Cognition Labs. Devin: The first AI software engineer. <https://devin.ai>, 2024.
- 446 CrewAI, Inc. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>, 2024.
- 448 Cursor. Cursor: The AI-first code editor. <https://cursor.sh>, 2024.
- 449 Cursor Research. Composer 2 technical report. *arXiv preprint arXiv:2603.24477*, 2026.
- 450 Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park,
451 Nitin Pasari, Chetan Rane, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering
452 tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- 453 Pengfei Gao, Zhao Tian, Xiangxin Meng, and Trae Research Team. Trae agent: An LLM-based agent for
454 software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.
- 455 Paul Gauthier. Aider: AI pair programming in your terminal. <https://github.com/paul-gauthier/aider>,
456 2023.
- 457 GitHub. GitHub Copilot: Your AI pair programmer. <https://github.com/features/copilot>, 2021.
- 458 Google. Agent Development Kit (ADK): An open-source framework for building AI agents. <https://google.github.io/adk-docs/>, 2025.
- 460 Google DeepMind. Gemini 3.1 Pro. <https://deepmind.google/models/gemini/pro/>, 2026.
- 461 Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu,
462 Y. K. Li, Fuli Luo, Yun Xiong, and Wenfeng Liang. DeepSeek-Coder: When the large language model meets
463 programming — the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.
- 464 Ahmed E. Hassan, Hao Li, Dayi Lin, Bram Adams, Tse-Hsun Chen, Yutaro Kashiwa, and Dong Qiu. Agentic
465 software engineering: Foundational pillars and a research roadmap. *arXiv preprint arXiv:2509.06216*, 2025.

466 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili
467 Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen
468 Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International*
469 *Conference on Learning Representations (ICLR)*, 2024.

470 Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama,
471 Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language
472 models for code. *arXiv preprint arXiv:2403.07974*, 2024.

473 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan.
474 SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on*
475 *Learning Representations (ICLR)*, 2024.

476 Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan,
477 Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and
478 Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The*
479 *Twelfth International Conference on Learning Representations*, 2024.

480 Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

481 LangChain. LangChain: Build context-aware reasoning applications. [https://github.com/langchain-ai/](https://github.com/langchain-ai/langchain)
482 [langchain](https://github.com/langchain-ai/langchain), 2022.

483 Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The rise of AI teammates in software engineering (SE 3.0):
484 How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*, 2025.

485 Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc
486 Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. StarCoder: May the source be with you! *Transactions*
487 *on Machine Learning Research (TMLR)*, 2023.

488 Seyedmoein Mohsenimofidi, Matthias Galster, Christoph Treude, and Sebastian Baltes. Context engineering for
489 AI agents in open-source software. *arXiv preprint arXiv:2510.21413*, 2025.

490 Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey
491 Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See,
492 Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog.
493 AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*,
494 2025.

495 OpenAI. Introducing Codex: A cloud-based software engineering agent. [https://openai.com/index/](https://openai.com/index/introducing-codex/)
496 [introducing-codex/](https://openai.com/index/introducing-codex/), 2025a.

497 OpenAI. OpenAI Agents SDK: A lightweight, powerful framework for multi-agent workflows. [https:](https://github.com/openai/openai-agents-python)
498 [//github.com/openai/openai-agents-python](https://github.com/openai/openai-agents-python), 2025b.

499 OpenAI. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>, 2026.

500 OpenClaw AI. OpenClaw: Personal AI assistant. <https://openclaw.ai>, 2025.

501 Thanosan Prathifkumar, Noble Saji Mathews, and Meiyappan Nagappan. Does SWE-Bench-Verified test agent
502 ability or model memory? *arXiv preprint arXiv:2512.10218*, 2025.

503 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng
504 Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for
505 software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational*
506 *Linguistics (ACL)*, 2024.

507 Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint*
508 *arXiv:2504.15228*, 2025.

509 Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. smola-
510 agents: A smol library to build great agentic systems. <https://github.com/huggingface/smolagents>,
511 2025.

512 Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu
513 Liu, Tal Remez, Jérémy Rapin, et al. Code Llama: Open foundation models for code. *arXiv preprint*
514 *arXiv:2308.12950*, 2023.

515 Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language
516 agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*,
517 2023.

518 Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. Finding widespread cheating on
519 popular agent benchmarks. Blog post, <https://debugml.github.io/cheating-agents/>, 2026a.

520 Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. Detecting safety violations across
521 many agent traces. *arXiv preprint arXiv:2604.11806*, 2026b.

522 TÂCHES. Get Shit Done: A light-weight meta-prompting, context engineering and spec-driven development
523 system for AI coding agents. <https://github.com/gsd-build/get-shit-done>, 2025. Initial commit
524 December 2025.

525 Hao Wang, Qiuyang Mang, Alvin Cheung, Koushik Sen, and Dawn Song. We scored 100% on AI bench-
526 marks without solving a single problem. Blog post, [https://moogician.github.io/blog/2026/](https://moogician.github.io/blog/2026/trustworthy-benchmarks/)
527 [trustworthy-benchmarks/](https://moogician.github.io/blog/2026/trustworthy-benchmarks/), 2026.

528 Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. AI agentic programming: A survey of
529 techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126*, 2025.

530 Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhi Li, Hao Peng, and Heng Ji. OpenHands: An
531 open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

532 Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and
533 Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural*
534 *Information Processing Systems (NeurIPS)*, 2022.

535 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun
536 Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen:
537 Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

538 Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based
539 software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

540 John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. SWE-agent:
541 Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

542 Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan.
543 Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information*
544 *Processing Systems (NeurIPS)*, 2023a.

545 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Syner-
546 gizing reasoning and acting in language models. In *International Conference on Learning Representations*
547 *(ICLR)*, 2023b.

548 Z.ai. GLM-5.1: Towards long-horizon tasks. Technical blog, <https://z.ai/blog/glm-5.1>, 2026.

A Full system prompt details

The system prompt is a structured document that governs the agent’s behavior across all tasks. It is not a generic instruction to “be helpful” but rather a precise specification of an engineering discipline. We describe its key sections.

A.1 Execution Mindset

The prompt opens with a directive that sets the tone:

```
# FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL.  
# BE RELENTLESS. BE CALM. BE RIGOROUS. BE ACCURATE. CHECK FACTS. NO AI SLOP.
```

Each directive addresses a specific failure mode observed in LLM-based agents:

“FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL.” LLMs have a tendency to drift: they comment on the difficulty of a problem, explore tangential concerns, or ask clarifying questions that the user has already answered. This directive anchors the model on the task at hand and discourages meta-commentary that consumes tokens without making progress.

“BE RELENTLESS.” When an agent encounters an error—a failing test, a type violation, a command that returns unexpected output—the default LLM behavior is to apologize, summarize the failure, and ask the user what to do next. This directive instructs the model to treat errors as obstacles to overcome, not as reasons to stop.

“BE CALM.” The complement of relentlessness. An overly aggressive agent that panics on encountering an error may thrash between approaches without deliberation. This directive encourages the model to analyze errors methodically before reacting.

“BE RIGOROUS.” This instructs the model to follow the verification and testing disciplines encoded later in the prompt, rather than taking shortcuts when a solution *looks* right.

“BE ACCURATE.” LLMs frequently generate plausible-but-wrong code, file paths, or command-line flags. This directive raises the model’s threshold for asserting facts, encouraging it to verify claims against the actual file system or documentation rather than relying on parametric memory.

“CHECK FACTS.” A more specific version of “be accurate” for information collected by KS Assistant uses web tools.

“NO AI SLOP.” “AI slop” refers to the low-quality, generic, hedging text that LLMs produce when they lack confidence: filler phrases like “certainly!”, unnecessary caveats, and boilerplate explanations that the user did not ask for. This directive discourages such output and encourages the model to be concise and substantive.

A.2 Tool Rules

Tool usage rules are explicit and mechanical:

```
# Rules  
- PWD = current working directory. Write() for new files; Edit() for small changes.  
- Run Bash synchronously with ‘timeout_seconds’ (default 300s). Retry with higher timeout on timeout. For  
  >10 min commands, run in background, redirect output to file, poll periodically.  
- Use go_to_url() for browser. Search the internet extensively.  
- **User only sees the finish() summary. Include full details/results/outputs. Never include meta-  
  descriptions like "Answered the user’s question about X" or "Fixed the bug in Y".**  
- Read large files in chunks. Temp files in PWD/tmp; clean up after.  
- Use ULTRA thinking ALWAYS.  
- **If running out of context/steps, don’t rush---call finish(is_continue=true).**
```

Each tool rule addresses a specific failure mode:

“PWD = current working directory.” The acronym “PWD” appears throughout the prompt and in user task descriptions (e.g., “edit PWD/src/main.py”). Without this definition, the model might interpret PWD as the Unix environment variable \$PWD and attempt to expand it, or misinterpret it as a literal directory name. The explicit definition ensures consistent interpretation.

602 **“Write() for new files; Edit() for small changes.”** Without this distinction, the model may use
603 Write() to overwrite an existing file with a slightly modified version, losing content it forgot to
604 include. By reserving Write() for new files and requiring Edit() for modifications, the instruction
605 ensures that changes are surgical and that unchanged portions of a file are never at risk.

606 **Bash timeout guidance.** LLMs frequently launch shell commands without considering their runtime.
607 A compilation or test suite that takes five minutes will time out at the default 30-second shell timeout
608 in most agent frameworks, causing spurious failures. The instruction to use 300 seconds as the
609 default, retry with higher timeouts on timeouts, and run long-running commands in the background
610 with output redirected to a file provides a mechanical protocol that handles common cases without
611 requiring the model to estimate runtime from first principles.

612 **“Use go_to_url() for browser. Search the internet extensively.”** The agent has access to multiple
613 tools that could plausibly interact with the web (shell-based curl, a Python program, the browser
614 tool, etc.). The first clause eliminates ambiguity by specifying which tool to use for browser-
615 based interactions. The second clause encourages the agent to proactively search the web when
616 it encounters unfamiliar APIs, libraries, or error messages, rather than relying on potentially stale
617 parametric memory.

618 **“User only sees the finish() summary.”** Users can get lost in the detailed trajectory generated by KS
619 Assistant in the chat window. Often users want to see only the final summary returned by the finish
620 tool, not the intermediate chain-of-thought or tool calls. Without this instruction, the model may
621 “tell” the user something in an intermediate message and then assume the user has seen it, leading to
622 confusion when the user asks for information the model believes it already provided. The companion
623 clause “Never include meta-descriptions” prevents the model from returning vague summaries like
624 “Fixed the bug in Y” instead of showing the actual fix; it forces the model to include concrete details,
625 results, and outputs in the summary.

626 **“Read large files in chunks. Temp files in PWD/tmp; clean up after.”** Reading a 10,000-line
627 file in a single tool call consumes a large fraction of the context window. By instructing the model
628 to read files in chunks, the prompt prevents context window exhaustion caused by a single-file
629 read, preserving capacity for the rest of the task. Without the temporary-file directive, the model
630 creates temporary files in unpredictable locations (the system /tmp, the home directory, or scattered
631 throughout the project). Centralizing temporary files in a known directory makes cleanup predictable,
632 and the explicit cleanup directive prevents the project tree from being polluted with stale artifacts
633 after the task completes.

634 **“Use ULTRA thinking ALWAYS.”** This instruction activates the model’s extended reasoning mode
635 (also known as “thinking” or “chain-of-thought” mode), in which the model performs additional
636 internal deliberation before producing a response. Extended reasoning is particularly valuable for
637 complex, multi-step tasks in which the model must plan before acting.

638 **“If running out of context/steps, don’t rush—call finish(is_continue=true).”** When the context
639 window is nearly full, LLMs exhibit a “rush to finish” behavior: they skip verification steps, make
640 hasty edits, and call finish with an incomplete result. This instruction redirects that urgency into
641 the continuation protocol (Section 2.2), ensuring that a clean handoff to a new sub-session produces
642 better results than a frantic attempt to squeeze everything into the remaining tokens.

643 A.3 Pre-flight Checks

644 Before modifying any file, we instruct the agent to read it first.

```
645 ## Pre-flight Checks
646
647 - Read every file before modifying it. Read relevant sources if the task depends on existing architecture.
648 - If referenced files/commands/config don't exist, stop and ask or report---don't guess.
649 - **When fixing bugs/issues/races: write tests to confirm first, then fix.**
```

652 Each pre-flight check targets a specific category of avoidable error:

653 **“Read every file before modifying it.”** The most common source of agent-introduced bugs is
654 modifying a file based on an incorrect assumption about its current contents. The model may
655 “remember” an older version of the file from its training data, or it may extrapolate from a partial
656 reading. By requiring a fresh read immediately before any edit, the instruction ensures that the model

operates on the file’s current state rather than a stale mental model. The companion clause “Read relevant sources if the task depends on existing architecture” extends this rule from individual files to architectural context. A task like “add a caching layer to the database module” requires understanding not just the file to be modified, but also how callers interact with it, what interfaces it provides, and what invariants it assumes. The instruction prevents the model from jumping straight to code generation without understanding the broader context.

“If referenced files/commands/config don’t exist, stop and ask or report—don’t guess.” LLMs have a strong tendency to confabulate: when asked to modify a file that does not exist, the model will often proceed as if it does, producing edits against phantom content. This instruction converts a silent failure (incorrect edits applied to a nonexistent file, which silently creates it) into an explicit clarification request.

“When fixing bugs/issues/races: write tests to confirm first, then fix.” This instruction mandates a test-first discipline for bug fixes. The motivation is two-fold: first, a test that reproduces the bug provides concrete verification that the fix is correct (the test should pass after the fix and fail before it). Second, writing the test forces the model to understand the bug precisely before attempting a fix, reducing the risk of an ad hoc patch that addresses a symptom rather than the root cause.

A.4 Code Style Guidelines

The prompt encodes a minimalist code philosophy:

```
## Code Style
- Simple, clean, readable code with minimal indirection. Organize in multiple files by functionality.
- Avoid unnecessary attributes, locals, config vars, tight coupling, and attribute redirections.
- DO NOT USE CLOSURES. No redundant abstractions or duplicate code.
- Public methods MUST have full documentation.
- Fix root causes, not symptoms. Think first: is the code simple, elegant, general, minimal?
- Don't write documentation unless the task requires it.
```

Each guideline addresses a specific anti-pattern commonly exhibited by LLM-generated code:

“Simple, clean, readable code with minimal indirection. Organize in multiple files by functionality.” LLMs tend to over-engineer solutions, introducing unnecessary abstractions, helper classes, and levels of indirection. Simple code is easier to review, test, and maintain. LLMs also often pile new code onto whichever file is currently being edited, producing 2,000-line modules that conflate unrelated concerns. This directive nudges the model toward a modular layout in which each file has a single, coherent responsibility.

“Avoid unnecessary attributes, locals, config vars, tight coupling, and attribute redirections.” LLMs frequently introduce intermediate variables that serve no purpose—for example, assigning a return value to a local variable only to immediately return it on the next line, or storing a constant in a configuration file when it is used in exactly one place. When the model adds a feature that touches multiple files, it may introduce imports, shared global state, or cross-module function calls that create tight coupling. An attribute redirection occurs when an object stores a reference to another object solely to forward method calls to it—for example, `self.x = other.x` at construction time, creating two paths to the same value. This single consolidated rule addresses all of these anti-patterns.

“DO NOT USE CLOSURES. No redundant abstractions or duplicate code.” LLMs reach for closures whenever a small piece of state needs to be carried alongside a function—producing nested defs that capture mutable variables from the enclosing scope. Such closures are difficult to test in isolation, opaque to type checkers, and a frequent source of subtle bugs due to the late binding of captured variables. The blanket prohibition steers the model toward explicit data structures (plain functions with arguments, classes with attributes), which are easier to reason about, easier to test, and play well with our no-mocks testing discipline. LLMs also sometimes create utility functions or classes that duplicate existing functionality; this instruction reminds the model to check for existing implementations before creating new ones.

“Public methods MUST have full documentation.” While the prompt generally discourages unnecessary documentation (see the last item), public methods are the API surface that other developers and modules depend on. Documentation on public methods is not optional—it specifies the contract.

712 **“Fix root causes, not symptoms. Think first: is the code simple, elegant, general, minimal?”**
 713 LLMs frequently apply symptom-level fixes: adding a null check where the real problem is that a
 714 variable should never be null, or catching an exception where the real problem is that the caller passes
 715 invalid arguments. This instruction forces the model to trace the causal chain to the root and fix it
 716 there. The companion metacognitive instruction asks the model to pause and evaluate its plan before
 717 committing to an implementation, spending more inference-time compute on design and reducing the
 718 likelihood of producing an unnecessarily complex first draft.

719 **“Don’t write documentation unless the task requires it.”** Claude Opus 4.6 tends to generate many
 720 documentation files. This instruction prevents the behavior.

721 A.5 Deep Work Rules

```
722 ## Deep Work
723
724 - For "align"/"match"/"make consistent": read the target state before editing. Never edit from vague
725   references.
726 - Use concrete values, not indirections (read Y first, then write specific values into X).
727 - List concrete planned changes before executing multi-part work.
728 - Every meaningful change needs a concrete verification method (test, grep, CLI).
```

731 The deep work rules address a failure mode where the model interprets an instruction loosely and
 732 makes changes that are directionally correct but concretely wrong:

733 **“For ‘align’/‘match’/‘make consistent’: read the target state before editing.”** When a user says
 734 “make file A consistent with file B,” the model often reads file A, infers what file B probably contains,
 735 and edits A based on that inference—without ever reading B. This instruction mandates reading the
 736 target first, ensuring that the alignment is based on concrete facts rather than assumptions.

737 **“Use concrete values, not indirections (read Y first, then write specific values into X).”** A related
 738 failure mode occurs when the model’s plan says “update X to match Y” but the model never resolves
 739 what Y actually is. The instruction requires the model to first read Y, extract the specific values, and
 740 then write those values into X. This eliminates a class of errors where the model’s mental model of Y
 741 differs from reality.

742 **“List concrete planned changes before executing multi-part work.”** When a task requires changes
 743 to multiple files, executing them one at a time without a plan leads to inconsistencies: the model may
 744 change a function signature in one file but forget to update a caller in another. Listing all planned
 745 changes before executing any of them forces the model to consider the full scope of the change and
 746 identify dependencies.

747 **“Every meaningful change needs a concrete verification method.”** A change without a verification
 748 method is a change that cannot be confirmed to work. This instruction requires the model to pair each
 749 change with a specific check—a test, a grep for the expected pattern, a CLI command that exercises
 750 the changed behavior—ensuring that the change can be validated programmatically rather than by
 751 visual inspection of a diff.

752 A.6 Planning for Complex Tasks

753 The planning instructions use a complexity threshold—three or more files, cross-module changes, or
 754 architectural work—to decide when formal planning is required:

```
755 ## Complex Task Planning
756
757 For 3+ files, cross-module, or architectural work:
758
759 1. List files to change and why.
760 1. State exact intended change per file.
761 1. Identify dependencies and execution order.
762 1. State verification method per change.
763
764 Skip for simple single-file tasks.
```

767 Each planning step targets a specific failure mode:

768 **“List files to change and why.”** This forces the model to enumerate the full blast radius of a change
769 before touching any file. Without this step, the model often discovers mid-task that additional files
770 need changes, leading to incomplete or inconsistent modifications.

771 **“State exact intended change per file.”** Listing files alone is insufficient; the model must also
772 articulate *what* will change in each file. This converts a vague plan (“update the database module”)
773 into a concrete specification (“add a `cache_ttl` parameter to `DatabaseClient.__init__`, modify
774 the query method to check the cache before hitting the database, add a cache invalidation method).

775 **“Identify dependencies and execution order.”** Some changes must precede others: a new utility
776 function must be written before callers can import it, a migration must run before code that depends
777 on the new schema. Identifying these dependencies prevents the model from applying changes in an
778 order that produces intermediate states where the code does not compile, or tests do not pass.

779 **“State verification method per change.”** The verification requirement from the Deep Work section
780 is reinforced here at the planning stage, ensuring that verification is planned alongside the changes
781 rather than treated as an afterthought.

782 The escape clause—“Skip for simple single-file tasks”—avoids the overhead of planning trivial
783 changes. Requiring a formal plan for a one-line typo fix would waste tokens and slow down the agent
784 without any compensating benefit.

785 A.7 Testing Instructions

786 The testing section is perhaps the most opinionated:

```
787 ## Testing
788
789 - Run lint/typecheckers; fix all errors. Achieve 100% branch coverage. Every error, including pre-existing
790   ones, is yours---don't skip.
791 - NO mocks, patches, fakes, or test doubles. Write integration/e2e tests. Each test independent, verifying
792   actual behavior.
793 - **Only run impacted tests after modifications.**
794 - To confirm races: add random sleep (<0.1s) before racing statements.
795
```

797 Each testing instruction addresses a specific concern:

798 **“Run lint/typecheckers; fix all errors. Achieve 100% branch coverage.”** Before committing any
799 change, the agent must ensure it does not introduce lint violations or type errors. This catches a
800 broad class of issues—unused imports, type mismatches, style violations—that would otherwise
801 accumulate across tasks. Full branch coverage ensures that every conditional path in the code under
802 test has been exercised. LLMs tend to write happy-path tests that cover the main code path but ignore
803 error handling, edge cases, and early-return branches. The 100% target pushes the model to write
804 tests for every branch, including error paths and boundary conditions. Such tests help with regression
805 by making failures visible before changes are accepted.

806 **“Every error, including pre-existing ones, is yours—don’t skip.”** Without this clause, the model
807 frequently rationalizes pre-existing lint or type errors as “not my problem” and calls `finish` with
808 a passing result despite a broken build. The instruction makes the agent responsible for the entire
809 codebase health, not just the delta it introduced.

810 **“NO mocks, patches, fakes, or test doubles. Write integration/e2e tests.”** This is the most
811 opinionated rule. Mock tests that verify code calls certain methods in a certain order test the
812 implementation, not the behavior. A test suite built on mocks can pass with flying colors while the
813 system is fundamentally broken, because the mocks hide the real dependencies. Integration tests that
814 exercise actual behavior are more expensive to run but provide genuine confidence that the system
815 works. Moreover, writing integration tests forces the model to think more deeply, often enabling the
816 agent to find deeper bugs in the code. The distinction between unit and integration tests matters: a
817 unit test in isolation may verify that a function produces the right output for a given input, but an
818 integration or end-to-end test verifies that the function works correctly within the larger system—with
819 real file I/O, real database connections, and real inter-module interactions.

820 **“Each test independent, verifying actual behavior.”** Test independence means that running tests in
821 any order produces the same results. Tests that depend on shared state or execution order are brittle

822 and difficult to debug when they fail. “Verifying actual behavior” reiterates that tests should assert on
823 observable outcomes (return values, side effects, system state) rather than implementation details.

824 **“Only run impacted tests after modifications.”** Running the full test suite after every small change
825 is wasteful when only a few modules are affected. For a large project, a full test run may take minutes,
826 and doing it after every edit adds up to significant wasted time and compute. This instruction directs
827 the model to identify which tests are affected by its changes and run only those, improving iteration
828 speed.

829 **“To confirm races: add random sleep (<0.1s) before racing statements.”** Race conditions are noto-
830 riously difficult to reproduce because they depend on precise timing. By inserting small sleep delays
831 at strategic points, the model can widen the race window and make the bug manifest deterministically
832 during testing. The 0.1-second upper bound keeps the test fast while still being sufficient to expose
833 most races.

834 A.8 Self-Improvement Loop

835 The agent maintains a preferences file that captures user invariants discovered during task execution:

```
836 ## Self-Improvement Loop
837
838
839 Read PWD/USER_PREFS.md at task start. Update with user preferences/invariants (no code snippets/symbols;
840 skip one-off tasks). Remove conflicting old entries carefully and thoroughly.
```

842 Each clause in this compact instruction serves the goal of cross-session learning:

843 **“Read PWD/USER_PREFS.md at task start.”** The preferences file contains invariants learned from
844 previous tasks—coding conventions, project-specific rules, architectural decisions. Reading it at the
845 start of each task ensures that the agent’s behavior is consistent across sessions, even though each
846 session starts with a fresh context window.

847 **“Update with user preferences/invariants.”** After completing a task, the agent may have learned
848 new information about the user’s preferences: a preferred naming convention, a disliked pattern, a
849 project-specific invariant. Writing these to the preferences file makes them available to future sessions.
850 This mechanism allows the agent to accumulate project knowledge over time without requiring the
851 user to repeat themselves.

852 **“No code snippets/symbols; skip one-off tasks.”** Code snippets in the preferences file are fragile:
853 they become stale as the codebase evolves, and they consume tokens that would be better spent
854 on natural-language descriptions of invariants. The companion rule about one-off tasks prevents
855 preference drift in the opposite direction: without it, the agent would record an entry every time it
856 completed any task, gradually polluting the file with idiosyncratic details (a particular file path, a
857 single throwaway experiment) that have no bearing on future work. Together, the two clauses keep
858 the file compact, robust to code changes, and focused on durable invariants.

859 **“Remove conflicting old entries carefully and thoroughly.”** Over time, preferences may become
860 contradictory—for example, an early preference might say “use camelCase for test methods” while a
861 later correction says “use snake_case.” Without explicit conflict resolution, the file would accumulate
862 contradictions, confusing the agent. This instruction mandates that the agent actively resolve conflicts
863 when updating the file to maintain internal consistency. *Note that we do not keep learnings in a*
864 *database or in various folders because it makes the information stale when lots of code changes are*
865 *happening. It is impossible for an agent to eliminate stale information across databases and multiple*
866 *folders.*

867 A.9 Pre-Finish Verification

868 Before declaring a task complete, the agent must pass a structured verification checklist:

```
869 ## Pre-Finish Verification
870
871
872 Before finish(success=True):
873
874 1. Re-read and verify every modified file.
875 1. Run required checks (lint, typecheck, tests); fix failures.
```

```

876 1. Check each user requirement against delivery.
877 1. If any check fails, keep working.
878 1. After 3 failed retries of same fix, rethink from scratch.

```

Each step in this checklist addresses a specific way agents declare premature success:

“Re-read and verify every modified file.” This is the analog of a code review performed by the agent on its own work. The model may have introduced a typo, forgotten to close a bracket, or made an edit that looked correct in the diff but was wrong in the full-file context. Re-reading the file after all edits are complete catches these errors.

“Run required checks (lint, typecheck, tests); fix failures.” This converts the subjective assessment “I think my changes are correct” into an objective, automated verification. If the lint, typecheck, or test suite fails, the agent must fix the failure before declaring success.

“Check each user requirement against delivery.” The model may have completed a task that it *thinks* satisfies the user’s request, but actually misses a requirement. This instruction forces a systematic comparison between the original task description and the delivered result, catching gaps and misinterpretations.

“If any check fails, keep working.” Without this instruction, the model may call `finish(success=True)` even when it knows a check has failed, rationalizing that the failure is “minor” or “unrelated.” The instruction makes the rule absolute: no finishing until all checks pass.

“After 3 failed retries of same fix, rethink from scratch.” LLMs can enter repetitive loops where they apply the same incorrect fix repeatedly, each time hoping for a different result. The three-retry threshold forces the model to break out of such loops by abandoning the current approach and reconsidering the problem from first principles. This is analogous to the debugging heuristic “if you’ve been staring at the same code for twenty minutes, you’re looking in the wrong place.”

A.10 Web Research Protocol

When the agent needs external knowledge, the prompt prescribes a structured research workflow rather than allowing ad-hoc browsing:

```

903 ## Web Research (MANDATORY)
904
905 - **Visit >=30 websites every search. Hard requirement---don't stop before 30 or rationalize fewer.**
906 - Procedure:
907   1. Create PWD/tmp/information-{unique_id}.md: '# Web Research --- Websites visited: 0/30'
908   1. Per site, append: '## [N/30] URL' + extracted info. Update header counter each visit.
909   1. **Don't proceed until counter >=30.**
910   1. If results dry up, try different queries, synonyms, official docs, GitHub repos/issues, Stack Overflow
911     , blogs, Reddit, papers, API refs.
912   1. After 30, review and think deeply.
913 - Ask user for login help when needed.
914

```

The rationale is a two-phase separation between *collection* and *synthesis*. LLMs tend to anchor on the first few results they encounter, which biases their solutions toward a narrow slice of the design space. By forcing the agent to accumulate a broad set of information into a file *before* reasoning about it, the protocol counteracts anchoring bias and encourages the model to consider diverse approaches. The “visit ≥ 30 websites” threshold is a conservative guardrail: it discourages the agent from stopping the research phase after two or three hits, and the resulting information file serves as an auditable artifact of what the agent considered. The structured procedure with a counter header (`# Web Research --- Websites visited: 0/30`) and per-site entries (`## [N/30] URL`) addresses an empirically observed failure mode in which the model claims to have “visited many sites” after only a handful of fetches; a concrete counter forces the model to verify the actual number visited before declaring the collection phase complete. The instruction to try “different queries, synonyms, official docs” when results dry up prevents the agent from giving up prematurely on a narrow set of search terms.

The login instruction addresses a practical obstacle in web research: many websites require authentication before revealing their content. Rather than silently skipping gated pages or hallucinating their contents, we instruct the agent to ask the user for help with login.

931 A.11 File Browsing Protocol

932 When a task requires understanding multiple source files before making changes, the prompt pre-
933 scribes the same two-phase collect-then-synthesize discipline used for web research, but applied to
934 the local file system:

```
935 ## File Browsing
936
937 Collect info and code snippets in PWD/tmp/file-information-{unique_id}.md without overthinking, then
938 review and think deeply.
939
```

941 This instruction addresses a failure mode distinct from the web research case. When an agent must
942 read many project files to understand a codebase before making changes, it tends to read a file, form a
943 hypothesis, and immediately begin editing—anchoring on the first few files it encounters and missing
944 relevant context in files it never opens. Worse, each file read consumes context window tokens; by
945 the time the agent has read enough files to understand the full picture, it may have already spent most
946 of its context window on the raw file contents, leaving little room for reasoning and code generation.

947 The file browsing protocol counteracts both problems. By writing a structured summary of each
948 file’s relevant information into a temporary markdown file, the agent externalizes its understanding
949 into a compact artifact that persists across context boundaries. The instruction to collect “without
950 overthinking” is deliberate: during the collection phase, the agent should extract and record facts
951 (function signatures, class hierarchies, call sites, invariants) rather than analyze or plan. Analysis
952 happens in the second phase, when the agent reads its own summary file and reasons about the
953 collected information as a whole.

954 This two-phase separation provides three benefits. First, it prevents premature commitment: the agent
955 cannot start editing until it has surveyed the relevant files, reducing the risk of changes that are locally
956 correct but globally inconsistent. Second, the summary file is typically much smaller than the raw
957 source files, freeing up context window capacity for subsequent reasoning and editing phases. Third,
958 the summary file serves as an auditable artifact—the developer can inspect it to verify that the agent
959 considered the right files and extracted the right information before making changes.

960 A.12 Desktop Application Control

961 The agent can interact with graphical desktop applications using screenshots, keyboard, and mouse:

```
962 ## Desktop Apps
963
964 Use screenshots, keyboard, and mouse. Don't launch VS Code or its extensions.
965
```

967 This instruction enables the agent to operate GUI applications (Preview, browsers, graphical diff tools)
968 when command-line alternatives are insufficient. The explicit prohibition on launching VS Code
969 prevents a recursive loop: since the agent runs *inside* a VS Code extension, launching another
970 VS Code instance or modifying extension state from within the agent could corrupt the host session
971 or create deadlocks. Note that modern LLMs support desktop control abilities, and we are merely
972 exploiting them.

973 A.13 Assistant-Specific Overrides

974 A final section provides project-specific instructions that are injected when the agent operates on its
975 own codebase:

```
976 ## Assistant-specific
977
978 - Lint/typecheck/format: 'uv run check --full'. Test: 'uv run pytest -v' (timeout 900s).
979 - **Do NOT install the KS Assistant extension from inside Assistant.**
980 - To open/edit system prompt: hidden for double-blind review
981 - KS Assistant info: hidden for double-blind review
982 - Third-party agents: KS/agents/third_party_agents
983 - Official Claude SKILLS: KS/agents/claude_skills
984 - Authenticate unauthenticated third-party agents; ask user only when a page needs auth.
985 - Read PWD/Assistant.md as overriding instructions.
986
```

988 These overrides serve seven purposes. First, they specify the exact toolchain commands for the
 989 KS project itself (`uv run check --full`, `uv run pytest`), eliminating guesswork about which
 990 linter, formatter, or test runner to use. Second, they prevent dangerous self-modification: just as
 991 a surgeon should not operate on themselves, the agent must not install or reinstall the VS Code
 992 extension it is running inside, since doing so would terminate its own process. Third, they provide
 993 navigation instructions for the agent’s own source: when the user asks to edit the system prompt, the
 994 agent knows the exact file path within the installed VS Code extension directory, avoiding guesswork
 995 about where the file lives. Moreover, it does not create unnecessary UI components in KS Assistant
 996 just to edit the system prompt. Fourth, the agent is given a URL to its own paper’s source as a
 997 source of self-knowledge: when a user asks the agent about its own capabilities, design, or identity, it
 998 can retrieve and read the paper rather than hallucinating an answer. This creates a self-referential
 999 loop in which the documentation describing the agent is also consumed by the agent. Fifth, the
 1000 instructions expose a third-party agent integration layer: the agent is told where third-party agents
 1001 live (`KS/agents/third_party_agents`). When a third-party agent requires authentication, the
 1002 agent handles it autonomously and only prompts the user when a page genuinely requires user
 1003 credentials—reducing unnecessary interruptions while maintaining security. Sixth, official Claude
 1004 SKILLS [Anthropic, 2025] are bundled in `KS/agents/claude_skills`, giving the agent access
 1005 to Anthropic’s curated skill library for common software engineering patterns; by indicating where
 1006 these skills reside, the agent can reference and apply them when relevant tasks arise. Seventh,
 1007 the `Assistant.md` override mechanism allows per-repository instructions to further customize the
 1008 agent’s behavior, forming a hierarchy: general system prompt → Assistant-specific instructions →
 1009 repository-specific `Assistant.md`. This approach avoids hard-coding repository-specific instructions
 1010 in the KS Assistant code. The `Assistant.md` file can include additional markdown files for further
 1011 instructions.

1012 B Conversational development case study (continued)

```

1013 ## Desktop Apps
1014
1015 Use screenshots, keyboard, and mouse. Don't launch VS Code or its extensions.
1016 
```

1018 The prohibition on launching VS Code prevents a recursive loop: since the agent runs inside a
 1019 VS Code extension, launching another instance could corrupt the host session or create deadlocks.

1020 **Step 3: Investigating unexpected post-merge state.** After testing the simplified flow, the developer
1021 notices that files appear in the Source Control panel of VS Code on the original branch after pressing
1022 “Commit and Merge” and asks:

1023 After the user presses "Commit and Merge", why do the modified files show up as
1024 committed in the original branch?
1025

1027 The agent traces the exact execution path through `squash_merge_branch()` and discovers that
1028 the implementation deliberately *unstages* the squash-merged changes via `git reset HEAD` so they
1029 appear as uncommitted working-tree modifications in VS Code’s Source Control panel. The design
1030 intention was to let the user review the diff before committing manually. The agent reports this
1031 finding along with the relevant code snippet and a verified end state showing no new commits on the
1032 branch.

1033 **Step 4: Directing a design change in one sentence.** The developer realizes that code review
1034 already happened while the worktree was active and decides the unstage step is unnecessary:

1035 The review is already happening in the worktree branch. You don’t need the user
1036 to review the modified and new files in the original branch. Fix it.
1037

1039 The agent replaces the `git reset HEAD` call in `squash_merge_branch()` with a conditional `git`
1040 `commit -no-edit` that uses the auto-generated squash message. It adds a `git diff -cached`
1041 `-quiet` guard for the edge case where the merge produces no changes. One existing test
1042 (`test_merge_leaves_changes_uncommitted`) is renamed to `test_merge_commits_changes`
1043 and its assertions are updated to verify a clean working tree. All 104 worktree tests pass.

1044 NeurIPS Paper Checklist

1045 1. Claims

1046 Question: Do the main claims made in the abstract and introduction accurately reflect the
1047 paper’s contributions and scope?

1048 Answer: [Yes]

1049 Justification: The abstract and introduction (Section 1) state the contributions—a five-layer
1050 agent architecture, a structured system prompt, a VS Code extension, and a 62.2% pass
1051 rate on Terminal-Bench 2.0—and limit the empirical scope to software engineering tasks
1052 evaluated on this benchmark with Claude Opus 4.6. The paper now treats Claude Code
1053 and Cursor Composer 2 numbers as public leaderboard context rather than a controlled
1054 head-to-head comparison. The self-hosting claim is presented as a development case study
1055 rather than causal evidence of performance.

1056 Guidelines:

- 1057 • The answer [N/A] means that the abstract and introduction do not include the claims
1058 made in the paper.
- 1059 • The abstract and/or introduction should clearly state the claims made, including the
1060 contributions made in the paper and important assumptions and limitations. A [No] or
1061 [N/A] answer to this question will not be perceived well by the reviewers.
- 1062 • The claims made should match theoretical and experimental results, and reflect how
1063 much the results can be expected to generalize to other settings.
- 1064 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1065 are not attained by the paper.

1066 2. Limitations

1067 Question: Does the paper discuss the limitations of the work performed by the authors?

1068 Answer: [Yes]

1069 Justification: Section 3 discusses consistently failed tasks and their failure categories (graph-
1070 ical/multimedia requirements, resource-intensive builds, niche domain knowledge). The
1071 conclusion acknowledges that results depend on a single frontier model (Claude Opus 4.6)
1072 and a single benchmark.

1073 Guidelines:

- 1074 • The answer [N/A] means that the paper has no limitation while the answer [No] means
1075 that the paper has limitations, but those are not discussed in the paper.
- 1076 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 1077 • The paper should point out any strong assumptions and how robust the results are to
1078 violations of these assumptions (e.g., independence assumptions, noiseless settings,
1079 model well-specification, asymptotic approximations only holding locally). The authors
1080 should reflect on how these assumptions might be violated in practice and what the
1081 implications would be.
- 1082 • The authors should reflect on the scope of the claims made, e.g., if the approach was
1083 only tested on a few datasets or with a few runs. In general, empirical results often
1084 depend on implicit assumptions, which should be articulated.
- 1085 • The authors should reflect on the factors that influence the performance of the approach.
1086 For example, a facial recognition algorithm may perform poorly when image resolution
1087 is low or images are taken in low lighting. Or a speech-to-text system might not be
1088 used reliably to provide closed captions for online lectures because it fails to handle
1089 technical jargon.
- 1090 • The authors should discuss the computational efficiency of the proposed algorithms
1091 and how they scale with dataset size.
- 1092 • If applicable, the authors should discuss possible limitations of their approach to
1093 address problems of privacy and fairness.
- 1094 • While the authors might fear that complete honesty about limitations might be used by
1095 reviewers as grounds for rejection, a worse outcome might be that reviewers discover
1096 limitations that aren’t acknowledged in the paper. The authors should use their best

1097 judgment and recognize that individual actions in favor of transparency play an impor-
1098 tant role in developing norms that preserve the integrity of the community. Reviewers
1099 will be specifically instructed to not penalize honesty concerning limitations.

1100 3. Theory assumptions and proofs

1101 Question: For each theoretical result, does the paper provide the full set of assumptions and
1102 a complete (and correct) proof?

1103 Answer: [N/A]

1104 Justification: The paper does not include theoretical results. It is a systems paper presenting
1105 an architecture and empirical evaluation.

1106 Guidelines:

- 1107 • The answer [N/A] means that the paper does not include theoretical results.
- 1108 • All the theorems, formulas, and proofs in the paper should be numbered and cross-
1109 referenced.
- 1110 • All assumptions should be clearly stated or referenced in the statement of any theorems.
- 1111 • The proofs can either appear in the main paper or the supplemental material, but if
1112 they appear in the supplemental material, the authors are encouraged to provide a short
1113 proof sketch to provide intuition.
- 1114 • Inversely, any informal proof provided in the core of the paper should be complemented
1115 by formal proofs provided in appendix or supplemental material.
- 1116 • Theorems and Lemmas that the proof relies upon should be properly referenced.

1117 4. Experimental result reproducibility

1118 Question: Does the paper fully disclose all the information needed to reproduce the main ex-
1119 perimental results of the paper to the extent that it affects the main claims and/or conclusions
1120 of the paper (regardless of whether the code and data are provided or not)?

1121 Answer: [Yes]

1122 Justification: Section 3 describes the benchmark (Terminal-Bench 2.0), the evaluation frame-
1123 work (Harbor), the model (Claude Opus 4.6), the hardware (2025 MacBook Air M4, 24 GB
1124 RAM), the number of trials (5 per task), and the 9 skipped tasks. The full agent architecture
1125 is described in Section 2 and the system prompt in Section 4 and Appendix A. Because the
1126 code is not included with the submission, reproduction would require reimplementing the
1127 described system or obtaining the released code later.

1128 Guidelines:

- 1129 • The answer [N/A] means that the paper does not include experiments.
- 1130 • If the paper includes experiments, a [No] answer to this question will not be perceived
1131 well by the reviewers: Making the paper reproducible is important, regardless of
1132 whether the code and data are provided or not.
- 1133 • If the contribution is a dataset and/or model, the authors should describe the steps taken
1134 to make their results reproducible or verifiable.
- 1135 • Depending on the contribution, reproducibility can be accomplished in various ways.
1136 For example, if the contribution is a novel architecture, describing the architecture fully
1137 might suffice, or if the contribution is a specific model and empirical evaluation, it may
1138 be necessary to either make it possible for others to replicate the model with the same
1139 dataset, or provide access to the model. In general, releasing code and data is often
1140 one good way to accomplish this, but reproducibility can also be provided via detailed
1141 instructions for how to replicate the results, access to a hosted model (e.g., in the case
1142 of a large language model), releasing of a model checkpoint, or other means that are
1143 appropriate to the research performed.
- 1144 • While NeurIPS does not require releasing code, the conference does require all submis-
1145 sions to provide some reasonable avenue for reproducibility, which may depend on the
1146 nature of the contribution. For example
 - 1147 (a) If the contribution is primarily a new algorithm, the paper should make it clear how
1148 to reproduce that algorithm.
 - 1149 (b) If the contribution is primarily a new model architecture, the paper should describe
1150 the architecture clearly and fully.

- 1151 (c) If the contribution is a new model (e.g., a large language model), then there should
1152 either be a way to access this model for reproducing the results or a way to reproduce
1153 the model (e.g., with an open-source dataset or instructions for how to construct
1154 the dataset).
- 1155 (d) We recognize that reproducibility may be tricky in some cases, in which case
1156 authors are welcome to describe the particular way they provide for reproducibility.
1157 In the case of closed-source models, it may be that access to the model is limited in
1158 some way (e.g., to registered users), but it should be possible for other researchers
1159 to have some path to reproducing or verifying the results.

1160 5. Open access to data and code

1161 Question: Does the paper provide open access to the data and code, with sufficient instruc-
1162 tions to faithfully reproduce the main experimental results, as described in supplemental
1163 material?

1164 Answer: [No]

1165 Justification: The repository link is withheld to respect double-blind review (stated in
1166 Section 1). No anonymized code package or reproduction scripts are currently provided
1167 with the submission.

1168 Guidelines:

- 1169 • The answer [N/A] means that paper does not include experiments requiring code.
- 1170 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/
1171 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 1172 • While we encourage the release of code and data, we understand that this might not
1173 be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not
1174 including code, unless this is central to the contribution (e.g., for a new open-source
1175 benchmark).
- 1176 • The instructions should contain the exact command and environment needed to run to
1177 reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- 1178 • The authors should provide instructions on data access and preparation, including how
1179 to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 1180 • The authors should provide scripts to reproduce all experimental results for the new
1181 proposed method and baselines. If only a subset of experiments are reproducible, they
1182 should state which ones are omitted from the script and why.
- 1183 • At submission time, to preserve anonymity, the authors should release anonymized
1184 versions (if applicable).
- 1185 • Providing as much information as possible in supplemental material (appended to the
1186 paper) is recommended, but including URLs to data and code is permitted.

1188 6. Experimental setting/details

1189 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
1190 rameters, how they were chosen, type of optimizer) necessary to understand the results?

1191 Answer: [Yes]

1192 Justification: Section 3 specifies the benchmark, model, hardware, number of trials, skipped
1193 tasks, and evaluation methodology. No training or fine-tuning is performed.

1194 Guidelines:

- 1195 • The answer [N/A] means that the paper does not include experiments.
- 1196 • The experimental setting should be presented in the core of the paper to a level of detail
1197 that is necessary to appreciate the results and make sense of them.
- 1198 • The full details can be provided either with the code, in appendix, or as supplemental
1199 material.

1200 7. Experiment statistical significance

1201 Question: Does the paper report error bars suitably and correctly defined or other appropriate
1202 information about the statistical significance of the experiments?

1203 Answer: [No]

1204 Justification: We report 5 independent trials per task (445 total runs), pass@any (78.7%),

1205 pass@all (43.8%), overall pass rate (62.2%), always-pass/always-fail/mixed-result counts,

1206 and median/mean cost and duration. However, the paper does not report confidence intervals,

1207 error bars, or statistical significance tests, so the checklist answer is No.

1208 Guidelines:

- 1209 • The answer [N/A] means that the paper does not include experiments.
- 1210 • The authors should answer [Yes] if the results are accompanied by error bars, confidence
- 1211 intervals, or statistical significance tests, at least for the experiments that support the
- 1212 main claims of the paper.
- 1213 • The factors of variability that the error bars are capturing should be clearly stated (for
- 1214 example, train/test split, initialization, random drawing of some parameter, or overall
- 1215 run with given experimental conditions).
- 1216 • The method for calculating the error bars should be explained (closed form formula,
- 1217 call to a library function, bootstrap, etc.)
- 1218 • The assumptions made should be given (e.g., Normally distributed errors).
- 1219 • It should be clear whether the error bar is the standard deviation or the standard error
- 1220 of the mean.
- 1221 • It is OK to report 1-sigma error bars, but one should state it. The authors should
- 1222 preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis
- 1223 of Normality of errors is not verified.
- 1224 • For asymmetric distributions, the authors should be careful not to show in tables or
- 1225 figures symmetric error bars that would yield results that are out of range (e.g., negative
- 1226 error rates).
- 1227 • If error bars are reported in tables or plots, the authors should explain in the text how
- 1228 they were calculated and reference the corresponding figures or tables in the text.

1229 **8. Experiments compute resources**

1230 Question: For each experiment, does the paper provide sufficient information on the com-

1231 puter resources (type of compute workers, memory, time of execution) needed to reproduce

1232 the experiments?

1233 Answer: [Yes]

1234 Justification: Section 3 specifies the hardware (2025 MacBook Air 15" with M4 processor

1235 and 24 GB RAM) and reports median/mean cost (\$0.45/\$0.90 per trial) and median/mean

1236 duration (202 s/446 s per trial).

1237 Guidelines:

- 1238 • The answer [N/A] means that the paper does not include experiments.
- 1239 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
- 1240 or cloud provider, including relevant memory and storage.
- 1241 • The paper should provide the amount of compute required for each of the individual
- 1242 experimental runs as well as estimate the total compute.
- 1243 • The paper should disclose whether the full research project required more compute
- 1244 than the experiments reported in the paper (e.g., preliminary or failed experiments that
- 1245 didn't make it into the paper).

1246 **9. Code of ethics**

1247 Question: Does the research conducted in the paper conform, in every respect, with the

1248 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

1249 Answer: [Yes]

1250 Justification: We have reviewed the NeurIPS Code of Ethics. The research involves building

1251 and evaluating a software engineering assistant; no human subjects, sensitive data, or

1252 dual-use concerns are involved.

1253 Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper focuses on architecture and benchmark evaluation. It does not include a dedicated broader-impact discussion covering both potential benefits and potential harms, so the checklist answer is No.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a pretrained language model or dataset. The system is an agent framework that wraps existing commercial LLM APIs.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

1308	12. Licenses for existing assets
1309	Question: Are the creators or original owners of assets (e.g., code, data, models), used in
1310	the paper, properly credited and are the license and terms of use explicitly mentioned and
1311	properly respected?
1312	Answer: [No]
1313	Justification: The paper cites the main tools, benchmarks, and models used, including
1314	Terminal-Bench 2.0, Harbor, Claude Opus 4.6, Playwright, VS Code, and related work. It
1315	does not yet explicitly list the license names or terms of use for each existing asset, so the
1316	checklist answer is No.
1317	Guidelines:
1318	• The answer [N/A] means that the paper does not use existing assets.
1319	• The authors should cite the original paper that produced the code package or dataset.
1320	• The authors should state which version of the asset is used and, if possible, include a
1321	URL.
1322	• The name of the license (e.g., CC-BY 4.0) should be included for each asset.
1323	• For scraped data from a particular source (e.g., website), the copyright and terms of
1324	service of that source should be provided.
1325	• If assets are released, the license, copyright information, and terms of use in the
1326	package should be provided. For popular datasets, paperswithcode.com/datasets
1327	has curated licenses for some datasets. Their licensing guide can help determine the
1328	license of a dataset.
1329	• For existing datasets that are re-packaged, both the original license and the license of
1330	the derived asset (if it has changed) should be provided.
1331	• If this information is not available online, the authors are encouraged to reach out to
1332	the asset’s creators.
1333	13. New assets
1334	Question: Are new assets introduced in the paper well documented and is the documentation
1335	provided alongside the assets?
1336	Answer: [No]
1337	Justification: The paper introduces a software system, but the anonymized code package
1338	and accompanying documentation are not included with the submission. No new datasets or
1339	pretrained models are released.
1340	Guidelines:
1341	• The answer [N/A] means that the paper does not release new assets.
1342	• Researchers should communicate the details of the dataset/code/model as part of their
1343	submissions via structured templates. This includes details about training, license,
1344	limitations, etc.
1345	• The paper should discuss whether and how consent was obtained from people whose
1346	asset is used.
1347	• At submission time, remember to anonymize your assets (if applicable). You can either
1348	create an anonymized URL or include an anonymized zip file.
1349	14. Crowdsourcing and research with human subjects
1350	Question: For crowdsourcing experiments and research with human subjects, does the paper
1351	include the full text of instructions given to participants and screenshots, if applicable, as
1352	well as details about compensation (if any)?
1353	Answer: [N/A]
1354	Justification: The paper does not involve crowdsourcing or research with human subjects.
1355	Guidelines:
1356	• The answer [N/A] means that the paper does not involve crowdsourcing nor research
1357	with human subjects.

- 1358
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- 1359
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.
- 1360
- 1361
- 1362
- 1363

1364 **15. Institutional review board (IRB) approvals or equivalent for research with human**

1365 **subjects**

1366 Question: Does the paper describe potential risks incurred by study participants, whether

1367 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

1368 approvals (or an equivalent approval/review based on the requirements of your country or

1369 institution) were obtained?

1370 Answer: [N/A]

1371 Justification: The paper does not involve human subjects research.

1372 Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

1383 **16. Declaration of LLM usage**

1384 Question: Does the paper describe the usage of LLMs if it is an important, original, or

1385 non-standard component of the core methods in this research? Note that if the LLM is used

1386 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

1387 scientific rigor, or originality of the research, a declaration is not required.

1388 Answer: [Yes]

1389 Justification: The entire system is built around LLM usage. The paper also discloses that the

1390 system was used to develop its own codebase (Section 1). An initial rough draft of the paper

1391 was generated from the code of KS Assistant and its README.md.

1392 Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.